Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу

«Операционные системы»

Группа: М8О-212БВ-25

Студент: Морозов Д. А.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 05.11.26

Москва, 2026

Постановка задачи

Вариант 12.

Наложить K раз фильтры эрозии и наращивания на матрицу, состоящую из вещественных чисел. На выходе получается 2 результирующие матрицы. Программа должна поддерживать последовательную и параллельную (многопоточную) реализацию с возможностью ограничения максимального числа одновременно работающих потоков.

Общий метод и алгоритм решения

В ходе работы были использованы следующие системные вызовы и механизмы:

- pthread_create(), pthread_join() — создание и ожидание потоков (POSIX Threads).
- malloc(), free() — динамическое выделение и освобождение памяти под матрицы.
- clock_gettime() — замер времени выполнения с высокой точностью.
- rand(), srand() — генерация случайных вещественных чисел для заполнения исходной матрицы.

Алгоритм фильтрации:
Для каждого элемента матрицы рассматривается окно 3x3 (текущий элемент и его 8 соседей).

- **Эрозия:** в окне ищется минимальное значение, которое записывается в результирующую матрицу.
- **Наращивание:** в окне ищется максимальное значение.

Поскольку операции применяются K раз, используется двойная буферизация (две пары матриц, которые меняются местами на каждой итерации), чтобы не перезаписывать исходные данные во время вычислений.

Параллельная реализация:
Матрица разбивается по строкам на p*p* частей, где p*p* — число потоков. Каждый поток независимо обрабатывает свой диапазон строк, записывая результат в свою область памяти. Так как потоки работают с разными строками, гонки данных не возникает, и использование мьютексов не требуется (что соответствует рекомендациям из лекции).

Код программы

seq.c

```
#include <stdio.h>

#include <stdlib.h>

#include <time.h>

double get_time() {

    struct timespec ts;
```

```
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec + ts.tv_nsec / 1e9;
}
double** alloc_matrix(int n) {
    double** mat = (double**)malloc(n * sizeof(double*));
    for (int i = 0; i < n; i++) mat[i] = (double*)malloc(n * sizeof(double));
    return mat;
}
void free_matrix(double** mat, int n) {
    for (int i = 0; i < n; i++) free(mat[i]);
    free(mat);
}
void fill_matrix(double** mat, int n) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            mat[i][j] = (double)(rand() % 256);
}
void copy_matrix(double** src, double** dst, int n) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            dst[i][j] = src[i][j];
}
void erode(double** src, double** dst, int n) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            double min_val = src[i][j];
            for (int di = -1; di <= 1; di++) {
                for (int dj = -1; dj <= 1; dj++) {
                    int ni = i + di, nj = j + dj;
                    if (ni >= 0 && ni < n && nj >= 0 && nj < n) {
```

```
                    if (src[ni][nj] < min_val) min_val = src[ni][nj];
                }
            }
        }
        dst[i][j] = min_val;
    }
  }
}
void dilate(double** src, double** dst, int n) {
  for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
      double max_val = src[i][j];
      for (int di = -1; di <= 1; di++) {
        for (int dj = -1; dj <= 1; dj++) {
          int ni = i + di, nj = j + dj;
          if (ni >= 0 && ni < n && nj >= 0 && nj < n) {
            if (src[ni][nj] > max_val) max_val = src[ni][nj];
          }
        }
      }
      dst[i][j] = max_val;
    }
  }
}
int main(int argc, char** argv) {
  if (argc < 3) {
    printf("Использование: %s <N> <K>\n", argv[0]);
    return 1;
  }
  int N = atoi(argv[1]);
```

```
    int K = atoi(argv[2]);
    srand(42);
    double** Orig = alloc_matrix(N);
    double** BufE = alloc_matrix(N);
    double** BufD = alloc_matrix(N);
    double** TempE = alloc_matrix(N);
    double** TempD = alloc_matrix(N);
    fill_matrix(Orig, N);
    copy_matrix(Orig, BufE, N);
    copy_matrix(Orig, BufD, N);
    double start_time = get_time();
    for (int k = 0; k < K; k++) {
        erode(BufE, TempE, N);
        double** swapE = BufE; BufE = TempE; TempE = swapE;
        dilate(BufD, TempD, N);
        double** swapD = BufD; BufD = TempD; TempD = swapD;
    }
    double end_time = get_time();
    printf("Время выполнения: %f секунд\n", end_time - start_time);
    free_matrix(Orig, N);
    free_matrix(BufE, N);
    free_matrix(BufD, N);
    free_matrix(TempE, N);
    free_matrix(TempD, N);
    return 0;
}
```

par.c

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
```

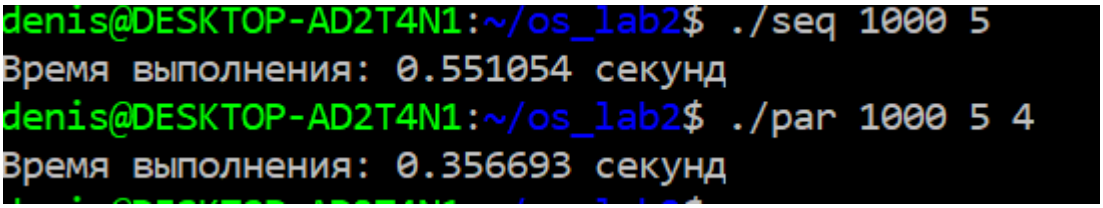
```
#include <pthread.h>
double get_time() {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec + ts.tv_nsec / 1e9;
}
double** alloc_matrix(int n) {
    double** mat = (double**)malloc(n * sizeof(double*));
    for (int i = 0; i < n; i++) mat[i] = (double*)malloc(n * sizeof(double));
    return mat;
}
void free_matrix(double** mat, int n) {
    for (int i = 0; i < n; i++) free(mat[i]);
    free(mat);
}
void fill_matrix(double** mat, int n) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            mat[i][j] = (double)(rand() % 256);
}
void copy_matrix(double** src, double** dst, int n) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            dst[i][j] = src[i][j];
}
typedef struct {
    double** src;
    double** dst;
    int n;
    int start_row;
```

```
    int end_row;
    int is_erode;
}
ThreadData;
void* worker(void* arg) {
    ThreadData* data = (ThreadData*)arg;
    int n = data->n;
    for (int i = data->start_row; i < data->end_row; i++) {
        for (int j = 0; j < n; j++) {
            double val = data->src[i][j];
            for (int di = -1; di <= 1; di++) {
                for (int dj = -1; dj <= 1; dj++) {
                    int ni = i + di, nj = j + dj;
                    if (ni >= 0 && ni < n && nj >= 0 && nj < n) {
                        if (data->is_erode) {
                            if (data->src[ni][nj] < val) val = data->src[ni][nj];
                        } else {
                            if (data->src[ni][nj] > val) val = data->src[ni][nj];
                        }
                    }
                }
            }
            data->dst[i][j] = val;
        }
    }
    return NULL;
}
int main(int argc, char** argv) {
    if (argc < 4) {
        printf("Использование: %s <N> <K> <num_threads>\n", argv[0]);
```

```
    return 1;
}
int N = atoi(argv[1]);
int K = atoi(argv[2]);
int num_threads = atoi(argv[3]);
srand(42);
double** Orig = alloc_matrix(N);
double** BufE = alloc_matrix(N);
double** BufD = alloc_matrix(N);
double** TempE = alloc_matrix(N);
double** TempD = alloc_matrix(N);
fill_matrix(Orig, N);
copy_matrix(Orig, BufE, N);
copy_matrix(Orig, BufD, N);
pthread_t* threads = (pthread_t*)malloc(num_threads * sizeof(pthread_t));
ThreadData* tdata = (ThreadData*)malloc(num_threads * sizeof(ThreadData));
int rows_per_thread = N / num_threads;
int extra = N % num_threads;
double start_time = get_time();
for (int k = 0; k < K; k++) {
    int current_row = 0;
    for (int t = 0; t < num_threads; t++) {
        int chunk = rows_per_thread + (t < extra ? 1 : 0);
        tdata[t].src = BufE;
        tdata[t].dst = TempE;
        tdata[t].n = N;
        tdata[t].start_row = current_row;
        tdata[t].end_row = current_row + chunk;
        tdata[t].is_erode = 1;
        current_row += chunk;
```

```
            pthread_create(&threads[t], NULL, worker, &tdata[t]);
        }
        for (int t = 0; t < num_threads; t++) pthread_join(threads[t], NULL);
        current_row = 0;
        for (int t = 0; t < num_threads; t++) {
            int chunk = rows_per_thread + (t < extra ? 1 : 0);
            tdata[t].src = BufD;
            tdata[t].dst = TempD;
            tdata[t].n = N;
            tdata[t].start_row = current_row;
            tdata[t].end_row = current_row + chunk;
            tdata[t].is_erode = 0;
            current_row += chunk;
            pthread_create(&threads[t], NULL, worker, &tdata[t]);
        }
        for (int t = 0; t < num_threads; t++) pthread_join(threads[t], NULL);
        double** swapE = BufE; BufE = TempE; TempE = swapE;
        double** swapD = BufD; BufD = TempD; TempD = swapD;
    }
    double end_time = get_time();
    printf("Время выполнения: %f секунд\n", end_time - start_time);
    free_matrix(Orig, N);
    free_matrix(BufE, N);
    free_matrix(BufD, N);
    free_matrix(TempE, N);
    free_matrix(TempD, N);
    free(threads);
    free(tdata);
    return 0;
}
```

Протокол работы программы



Число потоков	Время (сек)	Ускорение (S)	Эффективность (E)
1 (послед.)	0.551	1.00	1.00
2	0.358	1.54	0.77
4	0.318	1.73	0.43
16	0.308	1.79	0.11
128	0.398	1.38	0.01

Вывод

В ходе лабораторной работы были освоены принципы многопоточного программирования с использованием POSIX Threads. Реализованы последовательная и параллельная версии морфологических фильтров (эрозия и наращивание). Анализ замеров показал, что максимальное ускорение (1.79) достигается при 16 потоках. При увеличении числа потоков до 128 время выполнения растет из-за псевдопараллелизма и накладных расходов на переключение контекста, что подтверждает закон Амдала. Эффективность снижается с ростом числа потоков, поэтому оптимальным является использование количества потоков, равного числу ядер процессора.